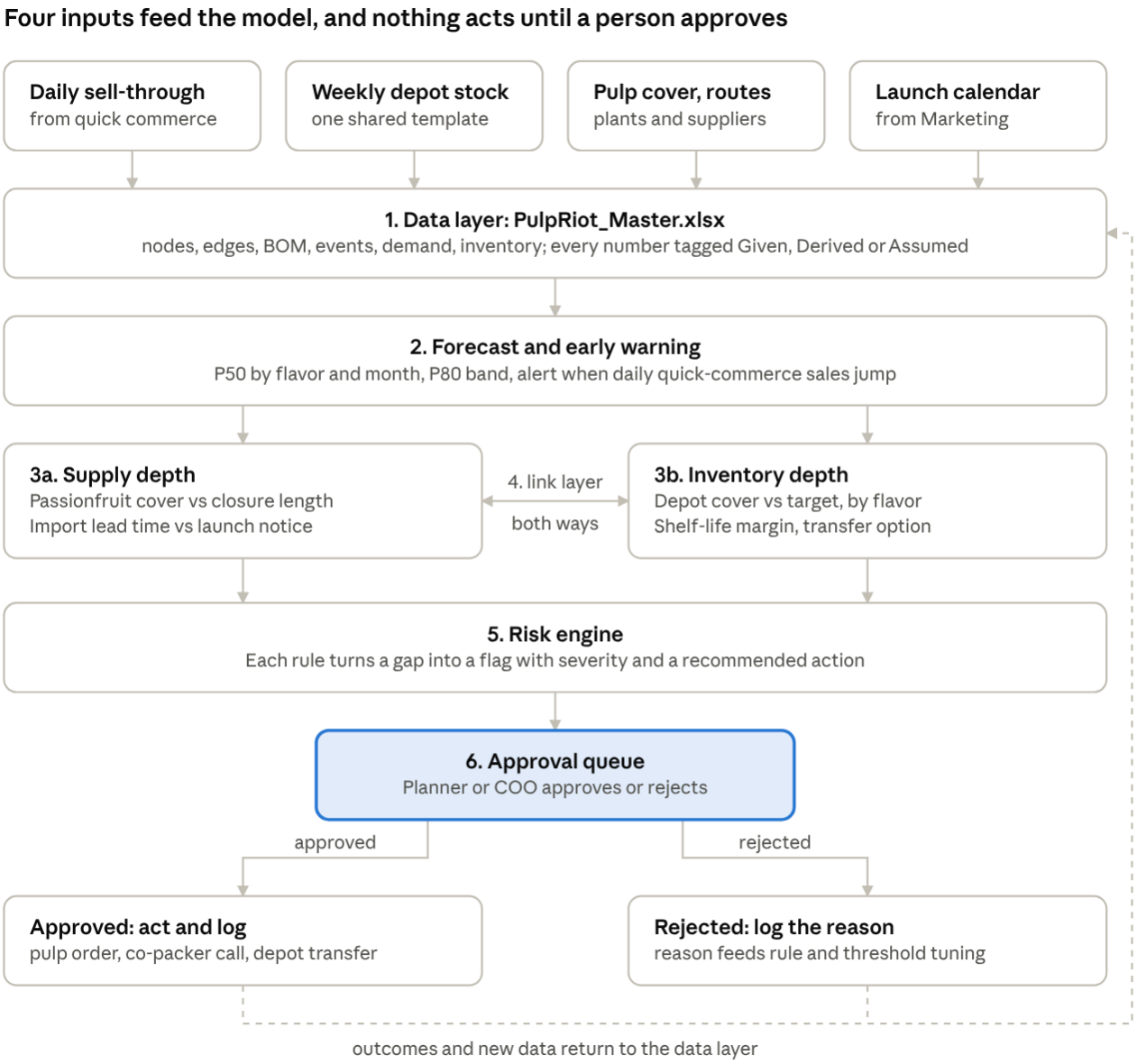
Pulp Riot: Model Build Workflow

For the model owner. How to build the Supply and Inventory model from PulpRiot_Master.xlsx using Claude.

Build the model in layers: prepare first, send Claude the right files, prompt one layer at a time, check every output, then fix in short rounds until it is final.

How the model works



model workflow · 7 stages, 1 approval gate

Data flows down the page once per cycle; the dashboard reads every stage, and the scenario replay runs last summer's events through stages 2 to 6 to show when each flag would have fired.

Stage	Takes in	Produces
1. Data layer	Daily sell-through, weekly depot stock, pulp cover and routes, launch calendar	One tagged dataset (Given, Derived, Assumed)
2. Forecast and early warning	Demand history, daily quick-commerce sales	P50 forecast, P80 band, demand-shift alert
3a. Supply depth	Pulp cover, closure length, import lead times, launch notice	Days of cover against risk, lead-time gaps
3b. Inventory depth	Depot stock, cover targets, shelf life	Depots above or below target, ageing risk, transfer options
4. Link layer	Outputs of 3a and 3b	Pulp shortage mapped to depot stock-out dates; depot demand mapped to pulp use
5. Risk engine	Stages 2 to 4	Flags with severity and a recommended action
6. Approval queue	Flags	Approve or reject decisions by a named person, then act and log

1. Before you start

Do these five things before the first prompt, because Claude builds what you describe and a vague spec gives a vague model.

1. **Know the case facts the model depends on.** Read the case PDF once and note the numbers every rule will use: 9.6 crore summer bottles, 2.5 crore a month plant capacity, 14 days of pulp cover against a 21-day closure, 3 weeks of depot cover, 60-day retailer cutoff, 30% forecast error.
2. **Write a one-page model spec.** List the inputs (workbook tabs), the calculations (forecast, cover, flags), the outputs (dashboard, flags, approval queue) and the user (a planner who approves each action). Keep it to one page so you can paste it into every prompt.
3. **Define "done" before you build.** Write the checks the model must pass, such as totals tying to the case and the replay flagging earlier than the real events. You will use this list in section 5.

4. **Fix the known workbook gaps first.** The workbook has no daily demand, so the viral spike is invisible; it does not reproduce the regional overstock; and its risk flags are static. Fix these in layer 1 of the build.
5. **Set up a Claude Project.** Upload the files in section 2 and add standing instructions, such as: use live formulas, never hardcode results; tag every number Given, Derived or Assumed; keep totals equal to the case figures; change only what I ask.

Keep one **control sheet** of reference numbers (9.6 crore bottles, ₹1,200 crore revenue, ₹46.08 crore lost revenue, ₹3.2 crore air freight, ₹6 crore write-offs, 38/20/16/14/12 flavor shares, 55/25/20 channel shares). Every check later compares against it.

2. What to send Claude

Send only what the layer needs, and always the newest version of each file. Name versions (v1, v2, v3) so you never build on a stale copy.

File	Why Claude needs it	When to send
Case PDF (MCKINSEY_PS_TF26.pdf)	Constraints: ₹2.5 crore budget, vendor IT, human approval, no general-trade data	Once, in the Project
PulpRiot_Master.xlsx (latest version)	The data foundation: nodes, edges, BOM, events, demand, inventory, flags	Every build prompt
supply_network.png	The map of flows and the three bottlenecks the model must cover	First prompt, and when building the dashboard
One-page model spec	Keeps Claude inside the scope you decided	Every build prompt
Control sheet of reference numbers	Lets Claude check its own totals	Every build prompt
Source CSVs and scripts from the Drive folder	Needed if you change how the flags are generated (build_graph.py, export_excel.py). Not opened yet, so confirm they match the workbook first	Only when changing data or flag logic

File	Why Claude needs it	When to send
Summary of the previous layer	Carries decisions into a fresh chat without the long history	Start of each new chat
The model file Claude just produced	Lets it fix its own output instead of rebuilding	Every fix prompt

Do not send the whole case and every file with each prompt. Extra material makes Claude drift, and it uses up your Claude Pro allowance faster.

3. How to write a prompt

A good build prompt has seven parts: role, context, task, inputs, rules, output format and checks. Ask for **one layer per prompt**, because a prompt that asks for everything produces a model you cannot verify.

```
ROLE: You are building an Excel planning model for Pulp Riot (case PS1).

CONTEXT: Model scope is Supply and Inventory in depth, with Demand,
Distribution and Transportation at a high level. Spec attached.

TASK: Build [one layer, e.g. the Forecast tab].

INPUTS: Use only the attached PulpRiot_Master.xlsx tabs [list tabs].
Do not change existing tabs.

RULES:
- Use live Excel formulas, never pasted results.
- Put every input and assumption in its own labelled cell.
- Tag every new number Given, Derived or Assumed, with a reason.
- Avoid XLOOKUP, FILTER, SORT, UNIQUE (they break on recalculation).
- Keep totals equal to the control sheet.

OUTPUT: One .xlsx file (v2), plus a change log of tabs added and
formulas used.

CHECKS: Before you finish, recalculate, report any formula errors,
and confirm the control totals. List anything you were unsure about.
```

A filled example for the forecast layer: "Add a Forecast tab. Use demand_monthly and channels. Show P50 demand by flavor and month, and a P80 band using the case's 30% forecast error as a labelled Assumed multiplier. Add a demand-shift flag that fires when daily quick-commerce sales stay above 2x baseline for 3 days. Do not touch other tabs."

Four habits that make prompts work:

- **Say what not to change.** "Do not edit existing tabs" protects your teammate's data.
- **Give the expected number.** "Summer total must equal 9.6 crore" lets Claude check itself.
- **Ask for the assumptions list.** Claude should name every choice it made, so you can review them.
- **Avoid "make it better."** Name the exact problem and the exact fix instead.

4. Build order

Build in this order, because each layer reads the one before it. Finish and check a layer before starting the next.

#	Layer	What it does	Reads from
1	Data fixes	Adds daily demand (Dragon Passion quick commerce rising 1x to 4x from 16 to 30 March) and a switchable regional overstock scenario	demand_monthly, events, channels
2	Forecast	P50 by flavor and month, a P80 band from the 30% error (labelled Assumed), and a demand-shift detector	Layer 1, products
3	Supply depth	Passionfruit pulp cover in days against closure length, import lead time against launch notice, shared-ingredient exposure	nodes, edges, bom
4	Inventory depth	Depot cover against the 21-day target, shelf-life margin to the 60-day cutoff, transfer option at ₹1.50 a bottle	inventory_snapshot, nodes
5	Link layer	Supply to Inventory: pulp running out becomes depot stock-out dates. Inventory to Supply: depot draw on a new flavor becomes pulp consumption and the allocation trade-off	Layers 2 to 4

#	Layer	What it does	Reads from
6	Risk engine	Turns each rule into a live flag with severity, recommended action and approver	Layers 2 to 5, risk_flags
7	Approval queue	Lists each recommendation as pending until a named person approves or rejects	Layer 6
8	Dashboard	One page across demand, supply, inventory, distribution and transportation	All layers
9	Scenario replay	Reruns the viral spike and the May closure and shows the first date each flag would fire against the real date	events, Layers 1 to 7
10	Loss and impact	Ties the ₹46.08 crore, ₹3.2 crore and ₹6 crore losses to the flags that would have prevented them	loss_ledger, Layer 9

For each layer, open a fresh chat, paste the model spec and the previous layer's summary, attach the latest workbook, and use the prompt template from section 3.

The **link layer (5)** is what the case asks for: it shows how a problem in one module causes a problem in the other, in both directions. Give it extra attention and test it hardest.

5. After each build: what to check

Claude can produce a clean-looking file with wrong numbers, so check every output yourself before moving on.

- [ ] Open the file and recalculate. There should be no #REF!, #NAME?, #VALUE! or #DIV/0! errors.
- [ ] Control totals still tie: 9.6 crore summer bottles, flavor shares 38/20/16/14/12, channel shares 55/25/20, ₹46.08 crore lost revenue.
- [ ] Click five output cells. Each should be a formula, not a typed number.
- [ ] Change one input (for example closure length from 21 days to 10) and confirm the dependent flags change.
- [ ] Work out three numbers by hand and match them to the sheet.
- [ ] Trace each flag to its evidence: it should point to a source row or case fact.
- [ ] Every new number is tagged Given, Derived or Assumed, with a reason.

- [ ] The original data tabs match version 1 exactly.
- [ ] Nothing is marked approved automatically. Every recommendation waits for a person.
- [ ] Nothing contradicts the case, for example output above the 2.5 crore a month plant capacity plus the co-packer's 10%.

Two extra checks for later layers:

- **Replay check:** the model's first flag date must come before the real event date (the planners' view on 31 March, the pulp running out on 25 May). If it does not, the rule or threshold is wrong.
- **Scale check:** add a sixth flavor row and a tenth depot row. Flags and totals should update without edits to formulas.

6. Fix loop until the model is final

Repeat this loop for every layer: build, check (section 5), write down each failure, send one fix prompt, then re-run all checks. Re-running the earlier checks matters, because a fix to one tab can break another.

What you find	How to ask for the fix
A total is wrong	"Cell [X] shows [value]. It should equal [expected] because [rule]. Find the cause and fix it, changing only that logic."
A number is typed, not calculated	"Convert [range] to formulas that read from [tab]. Do not change the values."
A flag fires too late or never	"The replay should flag by [date]. Show which rule and threshold decides this, then propose a threshold and explain the trade-off."
A flag fires too often	"Show how many flags fire per week. Propose a threshold that keeps only the ones a planner would act on."
Claude changed something you did not ask for	"Restore [tab] to v[n]. Re-apply only the change to [X]."
An assumption is hidden	"List every assumption in this tab, tagged Given, Derived or Assumed, with the reason."
The model is too slow or too heavy	"Remove [feature]. Keep the outputs listed in the spec."

Rules for the loop:

- **One change set per prompt.** Mixed requests make it hard to see what broke.
- **Save a new version after every accepted fix** (v2, v3, v4) and keep the last good one.
- **Send the file back** with each fix prompt, along with the failing check.
- **Start a fresh chat** when the conversation gets long. Paste the spec, a short summary of decisions made, and the latest file.
- **Keep a change log** in the workbook with date, change and reason, so teammates can follow what moved.

7. Definition of done

The model is final when every item below is true. Then freeze it and stop adding features.

- [ ] All ten layers are built and each passed the section 5 checks.
- [ ] The dashboard shows demand, supply, inventory, distribution and transportation, with depth on Supply and Inventory.
- [ ] The link layer shows both directions: pulp shortage to depot stock-outs, and depot demand to pulp use and the allocation trade-off.
- [ ] The replay flags the viral spike earlier than 31 March and the pulp shortfall before 11 May, with a recommendation at each flag.
- [ ] Every recommendation sits in the approval queue as pending, with a named approver.
- [ ] Changing one input (closure length, forecast error, cover target) updates flags without breaking anything.
- [ ] Adding a flavor, depot or channel row works without editing formulas.
- [ ] A README tab explains the colour legend (inputs, formulas, assumptions), the tab order and how to update the data.
- [ ] A change log and every Assumed value are visible in the workbook.
- [ ] The final file is saved as a locked copy (v-final) and a working copy.

Hand your teammates these model outputs: the flags table, the replay timeline, the loss and impact numbers, and the assumptions list. They need those for the root-cause and budget work.